

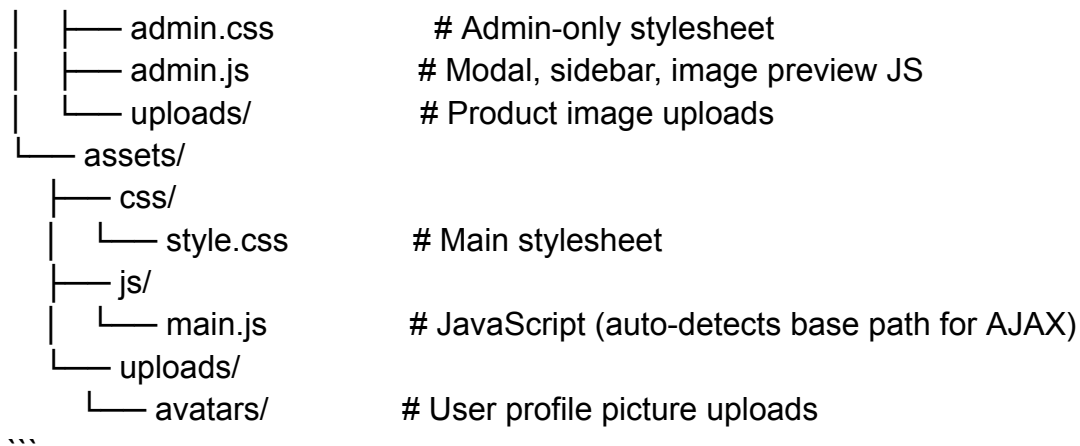
GadgetZone - Full-Featured E-Commerce Website

Build a fully functional, modern e-commerce platform for **gadgets** using PHP and MySQL. The system should include essential features such as product listings, user authentication, shopping cart, order management, and a responsive UI. Use high-quality product images sourced from <https://unsplash.com>

📁 File Structure

...

```
gadgetzone/
├── index.php           # Home page
├── database_setup.sql  # Database schema & sample data
├── migration_stripe_currency.sql # Migration for settings + Stripe columns
├── includes/
│   ├── db.php         # Database connection (starts session)
│   ├── functions.php  # Helper functions (auth, cart, formatting)
│   ├── currency.php   # Multi-currency config & formatPrice()
│   ├── header.php     # Shared header (navbar, search, cart badge)
│   └── footer.php     # Shared footer (cache-busted main.js)
├── pages/
│   ├── shop.php       # Product listing with filters & pagination
│   ├── product.php    # Product detail page
│   ├── cart.php       # Shopping cart (server-side remove fallback)
│   ├── cart_action.php # AJAX cart handler (add/update/remove)
│   ├── checkout.php   # Checkout (COD, bKash, Nagad, Stripe)
│   ├── stripe_checkout.php # Creates Stripe Checkout session
│   ├── stripe_return.php # Handles Stripe payment callback
│   ├── order_success.php # Order confirmation page
│   ├── myaccount.php  # User account dashboard (orders, profile, avatar)
│   ├── login.php      # Login page
│   ├── register.php   # Registration page
│   └── logout.php     # Logout handler
├── admin/
│   ├── index.php      # Admin dashboard (stats, recent orders)
│   ├── products.php   # Product CRUD management
│   ├── orders.php     # Order management & status updates
│   ├── users.php      # User & role management
│   ├── settings.php   # Currency & Stripe key configuration
│   ├── layout.php     # Shared admin sidebar (logo links to storefront)
│   └── footer.php     # Closing tags + admin.js
```



🏠 HOME PAGE PROMPT

DESIGN AESTHETIC:

- Dark theme background (**#0a0a0f**) with neon amber accents (**#f59e0b**)
- Typography: **IBM Plex Sans** font for headings (bold, 800 weight), **DM Sans** for body text
- Modern, editorial-style layout with generous spacing
- Smooth animations and hover effects

SECTIONS TO INCLUDE:

1. HERO SECTION:

- Split layout: Left side with headline, description, CTA buttons, stats
- Headline: **"Your World. Next-Level Technology."**
- Two CTAs: **"Shop Now"** (primary amber button) and **"Explore Deals"** (outline button)
- Stats row: **"500+ Products"**, **"50K+ Happy Customers"**, **"4.9★ Average Rating"**
- Right side: Large product image with floating badge showing **"Hot Deal Today - Up to 40% Off"**

2. FEATURE STRIP:

- Horizontal bar with 5 features: Free Delivery, 7-Day Returns, 2-Year Warranty, 24/7 Support, Secure Payment
- Each feature has an icon, title, and subtitle
- Background: slightly lighter than main background

3. CATEGORY GRID:

- 6 categories in a grid: Smartphones 📱, Laptops 💻, Audio 🎧, Cameras 📷, Wearables ⌚, Accessories 🔌
- Each card shows emoji icon, category name, and item count
- Hover effect: border changes to amber, slight lift animation

4. FEATURED PRODUCTS:

- Grid of 6 product cards
- Each card shows: image, category label, product name, star rating, price (current + old crossed out), "Add to Cart" button
- Product badge in top-left corner (NEW/HOT/SALE)
- Hover: card lifts up with amber glow shadow

5. DEAL OF THE DAY:

- Large banner with 3 columns: product info (left), product image (center), rating/delivery info (right)
- Countdown timer showing hours:minutes:seconds
- Large price display
- "Add to Cart" and "View Shop" buttons

6. NEW ARRIVALS:

- Similar product grid as Featured Products, showing 4 newest products

7. TESTIMONIALS:

- 3 customer testimonial cards in a row
- Each shows: 5-star rating, review text, customer avatar (circular with initials), name and location

8. NEWSLETTER:

- Full-width amber background section
- Centered content: headline "Get Exclusive Deals First 🎉", description, email input + subscribe button
- Input and button inline (rounded pill shape)

TECHNICAL REQUIREMENTS:

- PHP backend fetching products from MySQL database
- Products table structure: id, category_id, name, slug, description, price, old_price, image_url, badge (NEW/HOT/SALE), stock, featured, created_at
- Categories table: id, name, slug, icon
- Users table: id, first_name, last_name, email, password, phone, address, city, avatar, role, created_at

- `avatar VARCHAR(255)` stores filename of uploaded profile picture (stored in assets/uploads/avatars/)
- `role` ENUM: 'member', 'admin', 'super_admin'
- Settings table: id, setting_key, setting_value (stores active_currency, stripe_public_key, stripe_secret_key)
- Session-based cart stored in \$_SESSION['cart'] as [product_id => quantity]

CART IMPLEMENTATION (CRITICAL):

- Cart remove button MUST be wrapped in a <form method="POST"> — this is the server-side fallback
- cart.php handles POST with remove_id at the top BEFORE any HTML output, then redirects
- The remove form is ALSO intercepted by JS (form submit event, not button click) with AJAX for smooth UX
- If AJAX fails for any reason, the form naturally submits and removes the item server-side
- cart badge is ALWAYS in the DOM (not conditionally rendered), hidden via style="display:none" when count=0 — this allows JS to update it without a page reload

JAVASCRIPT BASE PATH (CRITICAL):

- main.js auto-detects environment at the top:
`const \_BASE = window.location.pathname.startsWith('/gadget') ? '/gadget' : ''`
- All AJAX calls use: fetch(_BASE + '/pages/cart_action.php', ...)
- This makes the same JS file work on localhost (/gadget/...) AND production VPS (root /)
- footer.php loads main.js with cache-busting: `

- Also run the sed command on main.js OR rely on the _BASE auto-detection (recommended)

- Responsive design for mobile/tablet

- Animations: fade-in on scroll for product cards, smooth hover transitions

- Product cards link to product detail page (pages/product.php?slug=...)

...

SHOP PAGE PROMPT

...

Create a complete shop/catalog page for an e-commerce website with filtering, sorting, and pagination:

LAYOUT:

- Two-column layout: Sidebar (260px) + Main content area

- Breadcrumb navigation at top

- Results count and sort dropdown in main header

SIDEBAR FILTERS:

- Sticky sidebar that stays visible on scroll

- Category filter: Radio buttons for All, Smartphones, Laptops, Audio, Cameras, Wearables, Accessories

- Price range slider: **Min \$0 to Max \$300,000** with live label update

- "Apply Filters" and "Clear All" buttons at bottom

- Dark surface background with subtle border

MAIN CONTENT:

- Header showing: "**Showing 1-9 of 45 results in Smartphones**"

- Sort dropdown: Newest, Most Popular, Top Rated, Price Low-High, Price High-Low

- Product grid (3 columns on desktop, responsive to 1 column on mobile)

- Same product card design as home page

- Empty state: Large search icon, "No products found" message, "Clear Filters" button

PAGINATION:

- Centered pagination controls at bottom

- Previous/Next arrow buttons

- Page number buttons (current page highlighted in amber)

TECHNICAL FEATURES:

- URL parameters:
?cat=smartphones&search=iphone&sort=price_asc&badge=SALE&page=2
- SQL query with WHERE filters, ORDER BY, LIMIT/OFFSET for pagination
- Form auto-submit on filter change
- Preserve filters (including badge) when sorting/paginating
- Product count per category shown in sidebar
- Sort options: Newest, Most Popular, Top Rated, Price Low-High, Price High-Low
- badge GET param filters by product badge (NEW/HOT/SALE) — used by "Explore Deals" button
- Product card images and names are clickable links to pages/product.php?slug=...

PHP LOGIC:

- Build dynamic WHERE clause from GET parameters
 - Calculate total pages: $\text{ceil}(\text{totalRecords} / \text{perPage})$
 - Fetch products with JOIN to categories table
 - Display results or empty state based on query results
- ...

CART PAGE PROMPT

...

Create a shopping cart page with dynamic quantity controls and order summary:

LAYOUT:

- Two-column: Cart items table (left) + Order summary sidebar (right, 360px)
- Breadcrumb: Home › Shopping Cart
- Page title with item count

CART ITEMS TABLE:

- Table header: Product | Price | Quantity | Subtotal | [Remove]
- Each row shows:
 - Product thumbnail (72x72px, rounded corners)
 - Product name and category
 - Unit price (bold, amber color)
 - Quantity controls: [-] [input field] [+] buttons
 - Subtotal (price × quantity)
 - Remove button (× icon)

- Hover effect: slight background highlight
- "Continue Shopping" button below table

QUANTITY CONTROLS:

- Minus button, number input (centered), plus button
- Min: 1, Max: 99
- AJAX update on change (no page reload)
- Updates subtotal and order total in real-time

ORDER SUMMARY (STICKY SIDEBAR):

- Section title: "Order Summary"
- Subtotal row
- Shipping row: "Free" if order > \$5,000, else \$150
- Progress message: "Add \$X more for free shipping!" (if under threshold)
- Total row (larger, bold, amber color)
- Coupon code input + Apply button
- "Proceed to Checkout" button (full width, large, primary amber)
- Payment icons at bottom: Visa, Mastercard, PayPal, Payoneer
- Security badge:  Secure Checkout Guaranteed

EMPTY CART STATE:

- Centered content: Shopping bag icon (large), "Your cart is empty" heading, description, "Start Shopping" button

AJAX FUNCTIONALITY:

- Add to cart from shop/home pages
- Update quantity
- Remove item (with fade-out animation)
- All cart operations update cart badge in header

PHP/SESSION:

- Cart stored in \$_SESSION['cart'] as array [product_id => quantity]
- Functions: getCart(), addToCart(), updateCartQty(), removeFromCart(), getCartCount(), getCartTotal()
- JSON responses for AJAX: {success: true, cart_count: 5, formatted_total: "₹59,999"}

CHECKOUT PAGE PROMPT

...

Create a multi-step checkout page with order review and payment options:

LAYOUT:

- Two-column: Checkout form (left) + Order review sidebar (right, sticky)
- Breadcrumb: Home › Cart › Checkout
- Three numbered sections in left column

SECTION 1: CONTACT INFORMATION

- Step number badge: "1" (amber circle)
- Section title: "Contact Information"
- Form fields in 2-column grid:
 - First Name * | Last Name *
 - Email Address * | Phone Number *
- Asterisk (*) indicates required

SECTION 2: SHIPPING ADDRESS

- Step number: "2"
- Title: "Shipping Address"
- Fields:
 - Street Address * (full width)
 - City * | Country (Bangladesh, readonly)
 - Order Notes (optional, textarea, 3 rows)

SECTION 3: PAYMENT METHOD

- Step number: "3"
- Title: "Payment Method"
- Radio button options (custom styled cards):
 - Cash on Delivery 💵
 - bKash (with bKash badge)
 - Nagad (with Nagad badge)
 - Credit/Debit Card 🏠
- Each option is a card with border, hover changes border to amber

ORDER REVIEW SIDEBAR (STICKY):

- Step badge: "✓"
- Title: "Order Review"
- List of cart items:
 - Each item: small thumbnail (56x56), product name, quantity, subtotal

- Summary rows:
 - Subtotal
 - Shipping (Free or ₺150)
 - Total (large, bold, amber)
- "Place Order" button showing final total: "Place Order – ₺59,999"
- Security message:  Your information is secure & encrypted

FORM VALIDATION:

- Server-side validation in PHP
- Error messages displayed at top in alert box
- Required field checks
- Email format validation
- Phone number format

ORDER PROCESSING:

- Generate unique order number: "GZ-" + uniqid()
- Insert into orders table: user_id, order_number, total_amount, status ('pending'), payment_method, shipping_address
- Insert order items into order_items table
- Clear cart session
- Redirect to order success page

PRE-FILL LOGIC:

- If user is logged in, pre-fill form with user data from database
- If not logged in, allow guest checkout

...

MY ACCOUNT PAGE PROMPT

...

Create a user account dashboard with sidebar navigation and multiple tabs:

LAYOUT:

- Two-column: Account sidebar (260px, sticky) + Content area
- Breadcrumb: Home › My Account

SIDEBAR:

- Top section (card with gradient background):

- Avatar circle with user initials (2 letters, amber background)
- User full name
- Email address
- Navigation menu (vertical list):
 -  Dashboard
 -  My Orders
 -  Profile
 -  Change Password
 -  Logout (red color)
- Active tab highlighted with amber background

TAB 1: DASHBOARD

- Welcome message: "Welcome back, [FirstName]! 🙌"
- 3 stat cards in a row:
 - Total Orders (count)
 - Delivered (count)
 - Total Spent (formatted price)
- Recent Orders section:
 - Table showing latest 5 orders
 - Columns: Order #, Date, Total, Status (badge), Payment
 - Status badges color-coded: pending (amber), processing (blue), shipped (purple), delivered (green), cancelled (red)

TAB 2: MY ORDERS

- Full orders table (all orders, not just 5)
- Includes "Items" column showing count
- Full timestamp with time
- Scrollable if many orders

TAB 3: PROFILE

- Form to update profile information:
 - First Name | Last Name (2 columns)
 - Email (disabled, with note "Email cannot be changed")
 - Phone Number
 - Address (textarea)
 - City
- "Save Changes" button
- Form pre-filled with current user data
- POST to same page, updates database, shows success message

TAB 4: CHANGE PASSWORD

- Form fields:
 - Current Password *
 - New Password * (min 6 characters)
 - Confirm New Password *
- Validation:
 - Check current password matches database hash
 - Check new password length
 - Check confirmation matches
- "Update Password" button
- Success/error messages

PHP LOGIC:

- requireLogin() function redirects to login if not authenticated
 - getCurrentUser() fetches user from database based on session
 - Handle form submissions with POST
 - Update user record in database
 - Password hashing with password_hash() and verification with password_verify()
 - Fetch orders with JOIN to get order items count
- ...

🔒 LOGIN & REGISTER PROMPTS

****LOGIN PAGE:****

...

Create a centered login form with:

- Card container (460px max width, dark surface, rounded corners, centered on page)
 - Title: "Welcome Back 🙌"
 - Subtitle: "Log in to your GadgetZone account"
 - Form fields:
 - Email Address (email input)
 - Password (password input)
 - "Log In" button (full width, large, amber primary button with lock icon)
 - Link at bottom: "Don't have an account? Create one →"
 - Error messages displayed in red alert box if login fails
 - Validation: check email exists, verify password hash
 - On success: set \$_SESSION['user_id'] and redirect to My Account or original page
- ...

****REGISTER PAGE:****

...

Create a registration form with:

- Similar card design as login
- Title: "Create Account 🚀"
- Subtitle: "Join GadgetZone and start shopping"
- Form fields:
 - First Name | Last Name (2 columns)
 - Email Address
 - Password (min 6 characters)
 - Confirm Password
- "Create Account" button
- Link: "Already have an account? Log in →"
- Validation:
 - Check all fields filled
 - Email format valid
 - Password minimum 6 characters
 - Passwords match
 - Email not already registered (query database)
- On success: insert user into database with `password_hash()`, auto-login, redirect to My Account

...

🎨 CSS Custom Properties

The design uses these CSS variables (defined in `:root``):

```
``css
--accent: #f59e0b;      /* Main amber color */
--accent-light: #fcd34d; /* Lighter amber for hovers */
--bg: #0a0a0f;          /* Main dark background */
--surface: #16161f;     /* Card backgrounds */
--border: rgba(255,255,255,0.08); /* Subtle borders */
--text: #f0f0f5;        /* Primary text */
--text2: #9090a8;       /* Secondary text */
--font-head: 'IBM Plex Sans'; /* Headings font */
--font-body: 'DM Sans';   /* Body text font */
```

```
--radius: 12px;          /* Border radius */  
...
```

🛠️ Key PHP Functions

```
**In `includes/functions.php`:**
```

```
```php  
isLoggedIn() // Check if user is logged in
requireLogin() // Redirect to login if not authenticated
getCurrentUser() // Get current user data from database
getCart() // Get cart array from session
addToCart($id, $qty) // Add product to cart
updateCartQty($id, $qty) // Update cart item quantity
removeFromCart($id) // Remove item from cart
getCartCount() // Total items in cart
getCartTotal() // Total price of cart
formatPrice($price) // Format as ₮ X,XXX
sanitize($data) // Clean input data
generateOrderNumber() // Create unique order ID
```
```

📱 Responsive Breakpoints

- **Desktop**: > 1024px (full layout)
- **Tablet**: 768px - 1024px (sidebars stack, 2-column grids)
- **Mobile**: < 768px (single column, hamburger menu, simplified tables)

🔒 Security Features

- Password hashing with `password\_hash()` and `PASSWORD\_DEFAULT`
- SQL injection prevention with `real\_escape\_string()` and prepared statements
- XSS prevention with `htmlspecialchars()` on all user input
- Session-based authentication
- CSRF protection (can be enhanced with tokens)

🚧 Future Enhancements

- Product search autocomplete
- Wishlist functionality

- Email notifications for orders
- Product reviews and ratings system
- Advanced filters (brand, price range slider)
- Image gallery for products

🇺🇸 Stripe Payment Integration

Database Migration

If you are setting up a fresh install, `database\_setup.sql` already includes everything.
For an existing database, run the migration:

```
```sql
USE gadgetzone;

-- Settings table (stores currency & Stripe keys)
CREATE TABLE IF NOT EXISTS settings (
 id INT AUTO_INCREMENT PRIMARY KEY,
 setting_key VARCHAR(100) NOT NULL UNIQUE,
 setting_value TEXT NOT NULL,
 updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP
);

-- Stripe columns on orders table
ALTER TABLE orders
 ADD COLUMN IF NOT EXISTS stripe_session_id VARCHAR(200) DEFAULT NULL,
 ADD COLUMN IF NOT EXISTS payment_status ENUM('unpaid','paid','refunded')
DEFAULT 'unpaid';

-- Default settings
INSERT INTO settings (setting_key, setting_value) VALUES
 ('active_currency', 'BDT'),
 ('stripe_publishable_key','pk_test_REPLACE_WITH_YOUR_KEY'),
 ('stripe_secret_key', 'sk_test_REPLACE_WITH_YOUR_KEY'),
 ('stripe_webhook_secret', '')
ON DUPLICATE KEY UPDATE setting_value = setting_value;
```
```

****Run via phpMyAdmin:**** Select `gadgetzone` DB → SQL tab → paste → Go.

****Run via CLI:****

```
```bash
mysql -u root -p gadgetzone < migration_stripe_currency.sql
```
```

Getting Stripe Sandbox Keys

1. Go to dashboard.stripe.com and sign in
2. Enable ****Test Mode**** (toggle top-right)
3. Go to ****Developers → API keys****
4. Copy your keys:
 - ****Publishable key**** — starts with `pk\_test\_`
 - ****Secret key**** — starts with `sk\_test\_`

Configuring Keys in Admin

1. Login at `/gadget/admin/index.php` (`admin@gadgetzone.com` / `Admin@1234`)
2. Click ****Settings**** in the sidebar
3. Paste your `pk\_test\_...` and `sk\_test\_...` keys in the Stripe section
4. Click ****Save All Settings****

Stripe Test Cards

| Card | Number | Exp | CVC |
|----------------|-----------------------|---------|--------|
| ----- | ----- | ----- | ----- |
| Visa (success) | `4242 4242 4242 4242` | `12/25` | `123` |
| Mastercard | `5555 5555 5555 4444` | `12/25` | `123` |
| Amex | `3782 822463 10005` | `12/25` | `1234` |
| Declined | `4000 0000 0000 0002` | `12/25` | `123` |

Payment Flow

1. Customer selects ****Credit / Debit Card**** at checkout
2. Form submits to `pages/stripe\_checkout.php` → creates a Stripe session
3. Customer is redirected to Stripe's hosted payment page
4. On success → `pages/stripe\_return.php` → verifies session → clears cart → `order\_success.php`

5. On cancel → redirected back to checkout with error message

Stripe Files

| File | Purpose |
|---------------------------------|---|
| `pages/stripe_checkout.php` | Creates Stripe Checkout session |
| `pages/stripe_return.php` | Handles post-payment callback |
| `pages/checkout.php` | Payment form (shows Stripe option when keys are configured) |
| `admin/settings.php` | UI for saving Stripe keys |
| `migration_stripe_currency.sql` | DB schema for settings + Stripe columns |

Troubleshooting

- **Stripe not showing at checkout** — Keys not saved or don't start with `pk\_test\_` / `sk\_test\_`
- **Invalid currency** — Go to Admin → Settings → select a supported currency
- **Payment not completed** — Session expired (valid 24 hrs); retry or use another method
- **Connection issues** — Ensure `curl` is enabled in PHP (`phpinfo()`); check firewall isn't blocking `api.stripe.com`

Multi-Currency Support

All prices are **stored in BDT (Bangladeshi Taka)** in the database and converted on the fly for display and Stripe.

Supported Currencies

| Code | Symbol | Name | Rate (approx) |
|------|--------|-------------------|---------------|
| BDT | ৳ | Bangladeshi Taka | 1.00 (base) |
| USD | \$ | US Dollar | ×0.0091 |
| EUR | € | Euro | ×0.0084 |
| GBP | £ | British Pound | ×0.0072 |
| CAD | C\$ | Canadian Dollar | ×0.0124 |
| AUD | A\$ | Australian Dollar | ×0.0140 |

| INR | ₹ | Indian Rupee | ×0.76 |
| SGD | S\$ | Singapore Dollar | ×0.0122 |
| SAR | ريال | Saudi Riyal | ×0.034 |
| AED | درهم | UAE Dirham | ×0.033 |
| JPY | ¥ | Japanese Yen | ×1.39 |
| MYR | RM | Malaysian Ringgit | ×0.042 |

Changing the Active Currency

1. Admin Dashboard → ****Settings****
2. Select a currency from the visual grid
3. Click ****Save All Settings****

The change takes effect site-wide immediately (session cache is invalidated on save).

Key Functions (includes/currency.php)

```
```php
getActiveCurrency(): array // Returns active currency array from DB-backed session
cache
formatPrice(float $bdt): string // Converts BDT → active currency and formats with
symbol
convertAmount(float $bdt): float // Converts BDT → active currency numeric value
getStripeAmount(float $bdt): int // Converts BDT to Stripe smallest unit (cents)
getStripeCurrencyCode(): string // Returns Stripe-compatible currency code (e.g. 'usd')
```
```

> ****Note for Bangladesh merchants:**** Stripe does not directly process BDT. When a customer pays by card, the amount is converted to the active display currency (default USD) for Stripe. COD, bKash, and Nagad payments are unaffected and always work in BDT.

📞 Support

For questions or issues, contact: support@gadgetzone.com

📄 License

This is a sample e-commerce project for educational purposes.

****Copy Right 2026 All rights reserved by Kitpapa.com****